

✓ AI Web Search v2 — Agent Loop with Tool Use

 [Open in Colab](#)

Input: User Query

Architecture: Agent Loop with Tool Use and Multiple LLM Calls

1. Define a `web_search` function as a **tool** the LLM can invoke via the Tavily API.
2. Run an **Agent Loop** (up to `MAX_LOOP_TIMES` LLM calls):
 - The LLM decides when and what to search.
 - Each search result is labeled `[n]`; results accumulate and are deduplicated across iterations.
 - Loop exits when the LLM returns an answer instead of a tool call.
3. Render the output as HTML in this notebook:
 - **Answer Section** — LLM answer with inline citations `[n] (URL)`
 - **Web Search Results Section** — all unique results in discovery order

```
# Install dependencies (run once)
%pip install -q litellm tavily-python markdown
```

```
===== 17.1/17.1 MB 70.8 MB/s eta 0:00:00
===== 278.1/278.1 kB 19.4 MB/s eta 0:00:00
```

```
import os, json, re
import litellm
from tavily import TavilyClient
from IPython.display import display, HTML
import markdown

# --- API Keys -----
TAVILY_API_KEY = "tvly-..." # your Tavily API key
OPENAI_API_KEY = "sk-..." # your OpenAI API key
os.environ["OPENAI_API_KEY"] = OPENAI_API_KEY

# --- Configuration -----
MODEL = "gpt-4o-mini" # change to e.g. "gpt-4o", "claude-3-5-haiku-20241022"
TOP_K = 10 # max web results per search
MAX_LOOP_TIMES = 3 # max LLM calls per request (including final answer)

litellm.set_verbose = False
```

✓ Step 1: Define Web Search as a Tool

`search_web` fetches results from Tavily. `WEB_SEARCH_TOOL` is the function schema exposed to the LLM so it can call this tool when needed.

```
def search_web(query, k=TOP_K):
    """Return top-k Tavily results for query."""
    response = TavilyClient(api_key=TAVILY_API_KEY).search(query, max_results=k)
    return response["results"] # each: {title, url, content, score}

WEB_SEARCH_TOOL = {
    "type": "function",
    "function": {
        "name": "web_search",
        "description": "Search the web for current information on a topic.",
        "parameters": {
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "The search query"}
            },
            "required": ["query"]
        }
    }
}
```

✓ Step 2: Agent Loop

The loop initializes the conversation with the user query, then repeatedly calls the LLM:

- If the LLM issues a `web_search` tool call, we execute the search, number the new results globally ([n]), deduplicate by URL, and add the results to the message context.
- If the LLM returns a plain text response (no tool call), we exit and return it as the answer.

The loop runs at most `MAX_LOOP_TIMES` times (= max total LLM calls).

```
SYSTEM_PROMPT = (
    "You are a helpful research assistant with access to a web_search tool. "
    "Use it to find information needed to answer the user's query; you may search multiple times with different quer
    "Each search result is labeled [n] with its URL. "
    "Cite supporting text in your answer as [n](URL) using those numbers. "
    "When you have enough information, write a clear, comprehensive markdown answer."
)

def run_agent(query, model=MODEL, max_loops=MAX_LOOP_TIMES):
    """Run the agent loop; return (answer_text, all_results, model_name)."""
    seen_urls = {} # url -> result dict (augmented with "num" key)
    counter = 0 # global discovery counter across all searches
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": query},
    ]

    last_content = ""
    model_name = model
    for loop_num in range(max_loops):
        print(f"\n[Loop {loop_num + 1}] Calling LLM...")
        response = litellm.completion(
            model=model, messages=messages,
            tools=[WEB_SEARCH_TOOL], tool_choice="auto"
        )
        msg = response.choices[0].message
        model_name = response.model # actual model name returned by the API
        last_content = msg.content or ""

        if not msg.tool_calls: # LLM produced a plain answer – exit loop
            print(f"[Loop {loop_num + 1}] LLM returned answer. Exiting loop.")
            return last_content, list(seen_urls.values()), model_name

        # --- Execute tool calls and add results to context ---
        messages.append(msg)
        for tc in msg.tool_calls:
            search_query = json.loads(tc.function.arguments)["query"]
            print(f"[Loop {loop_num + 1}] Tool call: web_search('{search_query}')"
            results = search_web(search_query)
            print(f"[Loop {loop_num + 1}] Got {len(results)} results")

            tool_text = ""
            for r in results:
                if r["url"] not in seen_urls: # deduplicate; first occurrence wins
                    counter += 1
                    r["num"] = counter
                    seen_urls[r["url"]] = r
                    num = seen_urls[r["url"]]["num"]
                    tool_text += f"[{num}] {r['title']}\nURL: {r['url']}\n{r['content']}\n\n"

            messages.append({
                "role": "tool", "tool_call_id": tc.id,
                "name": "web_search", "content": tool_text
            })

        # Max loops reached – return whatever the LLM last said
        print(f"[Loop {max_loops}] Max loops reached. Returning last LLM response.")
        return last_content, list(seen_urls.values()), model_name
```

✓ Step 3: Format and Display Output

`format_output` assembles the final markdown:

- **Answer** section: the LLM's response (with `[n] (URL)` citations).
- **Web Search Results** section: all unique results in discovery order (`[1]`, `[2]`, ...).

`display_answer` converts markdown to HTML and renders it in the notebook.

```
def format_output(answer_text, all_results, model_name):
    """Combine LLM answer and search results into a single markdown string."""
    # Normalize LLM citations: replace full-width 【 with standard [
    answer_text = answer_text.replace('【', '[')
    # Wrap citation numbers in brackets so [2](URL) renders as [2] instead of 2
    answer_text = re.sub(r'\s*([0-9]+)\s*\(', r'[{}](\s*'.format(model_name), answer_text)

    # Display results in ascending [n] (discovery) order
    ordered = sorted(all_results, key=lambda r: r["num"])

    results_md = "## Web Search Results\n\n"
    for r in ordered:
        results_md += f"- [{r['num']}] [{r['title']}]({r['url']})\n\n"

    return f"## Answer *(model: {model_name})*\n\n{answer_text}\n\n---\n\n{results_md}"

def display_answer(md_text):
    """Render markdown as HTML in the notebook."""
    html = markdown.markdown(md_text, extensions=["extra"])
    display(HTML(html))
```

✓ Run the Demo

Set your query below and run the cell. The agent will search the web as needed and display a cited answer.

```
query = "Claude Code 是如何实现的?" # <- change this

answer_text, all_results, model_name = run_agent(query)
output_md = format_output(answer_text, all_results, model_name)
display_answer(output_md)
```

```
[Loop 1] Calling LLM...
[Loop 1] Tool call: web_search('Claude Code 实现 研究')
[Loop 1] Got 10 results

[Loop 2] Calling LLM...
[Loop 2] Tool call: web_search('Claude Code implementation details')
[Loop 2] Got 10 results

[Loop 3] Calling LLM...
[Loop 3] LLM returned answer. Exiting loop.
```

Answer (model: gpt-4o-mini-2024-07-18)

Claude Code 是由 Anthropic 开发的一款先进编程助手，旨在通过综合的 AI 技术与工具集成来提升软件开发效率。以下是关于 Claude Code 如何实现的信息。

1. 系统架构和设计

Claude Code 的架构基于复杂的系统设计，主要涵盖以下几个层面：

- **用户接口层**：提供图形用户界面(GUI)和命令行界面 (CLI)，让开发者能够轻松与系统互动。
- **代理循环**：这是 Claude Code 的核心机制，通过持续查询处理和上下文管理来执行任务。
- **权限系统**：控制用户可以访问的功能和工具，确保安全和合规性。
- **工具管理**：支持多种工具和功能，包括文件编辑、编程语言支持、API 访问等。
- **状态与持久性**：管理系统的状态信息和会话持久性，以保证在多个会话之间共享上下文。

根据 Claude Code 的源码分析，超过 98% 的代码都是用于基础设施管理，其中包含了权限管理、上下文管理和工具路由逻辑等[1][12]。

2. 功能特性

Claude Code 集成了高度灵活的工具和功能，能够执行以下操作：

- **代码编辑和运行**：能够读取、编辑文件并执行命令，例如编译和运行测试。
- **上下文理解**：Claude Code 会分析代码上下文，并根据项目需求自动进行多次迭代。
- **任务驱动**：Claude 不仅响应单次请求，而是将问题视为可能需要长时间处理的循环任务。例如，当任务未完成时将自动续聊并继续执行[1][11]。
- **用户和项目记忆系统**：Claude Code 能持续学习用户的偏好和项目的背景，以改进未来的交互和决策质量[1]。

3. 实现技术

Claude Code 使用了 TypeScript 和现代开发框架构建，代码库接近 51 万行，涵盖大量的工具、UI 组件和执行引擎。其设计的核心是创建一个高效的查询引擎，能够处理复杂的任务和上下文管理，确保决策过程顺畅，并尽可能减少用户的手动输入[1][12]。

Claude Code 不仅是一个简单的编程工具，更是一种综合的工程产品，它通过连接不同的工具和资源来支持完整的开发工作流，旨在提高开发者的工作效率并减轻重复性工作负担。

总结

Claude Code 的成功在于其灵活的架构设计、智能的任务管理和高效的工具集成，使得开发者能在多个环境中快速实施、测试和优化代码。这一切都是为了建立一个能真正理解并适应用户的一站式编程助手。

Web Search Results

- [1] [深度解析Claude Code 51万行源码背后的设计实现 - 张永清 - 博客园](#)
- [2] [ThreeFish-AI/analysis_claude_code: Claude Code 逆向工程研究仓库](#)